Control Flow, Functions and the Stack

Introduction to ARM Cortex-M Microprocessors — Chapter 7

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

Table of Contents

1. Why This Matters
2. Making Decisions
3. Implementing C Control Structures
4. Subroutines
5. The Stack
6. Procedure Call Standard for the ARM Architecture (AAPCS)
7. Summary

Why This Chapter Matters

Every `if`, `while` and function call becomes the same two things: a comparison that sets flags, and a branch that tests them. This explains why a signed and an unsigned comparison can differ on the same bits, and why a recursive function can crash a microcontroller. It also prepares you for interrupts, which are function calls made by hardware.

After this chapter, you can:

1. Explain the `S` suffix and condition codes.
2. Branch on a condition with `CMP`, `TST` and suffixes.
3. Choose signed or unsigned condition codes correctly.
4. Translate C `if/else` and loops to assembly.
5. Trace SP through `PUSH` and `POP` on a full descending stack.
6. Call with `BL`, return with `BX LR`, and explain why nested calls save LR.
7. State the AAPCS rules.

© Muním Zabidi

By the end of this chapter you can read a C condition in assembly. Notice that the branch tests the opposite of the condition:

```
if (x == 5) { y = 1; }

    CMP   R0, #5      ; compare x with 5
    BNE   skip        ; opposite of ==
    MOVS  R1, #1      ; y = 1
skip
```

Questions this code raises, and where this chapter answers them:

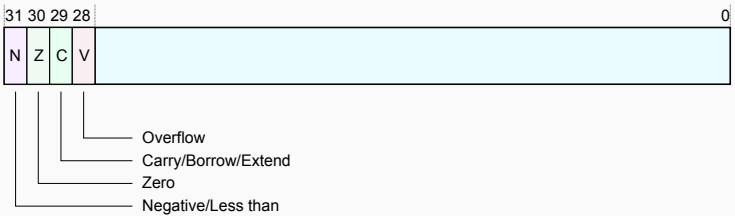
Question raised	Answered in
What does `CMP` change?	Making decisions: flags
Why `BNE`, not `BEQ`?	Condition codes
How does a function return?	Subroutines: `BL` and `BX LR`
Why must `LR` be saved?	The stack

© Muním Zabidi

Table of Contents

1. Why This Matters
2. **Making Decisions**
3. Implementing C Control Structures
4. Subroutines
5. The Stack
6. Procedure Call Standard for the ARM Architecture (AAPCS)
7. Summary

Processor Status Register



Flag	Description
N	1 means result is negative
Z	1 means result is zero
C	unsigned add: 1 means result had a carry out; unsigned subtract: 0 means it had a borrow; shifts: the bit shifted out
V	signed add/subtract: 1 means the result overflowed

Setting the Flags

The S suffix

```
ADD  R0, R1, R2   ; flags unchanged

ADDS R0, R1, R2   ; updates N Z C V
```

Compare and test — no result kept

Instruction	Computes
`CMP Rn, Op2`	Rn − Op2
`CMN Rn, Op2`	Rn + Op2
`TST Rn, Op2`	Rn & Op2
`TEQ Rn, Op2`	Rn ^ Op2

Why compilers care about S

A compiler writes `SUBS` instead of `SUB` when a branch right after it needs the flags. So seeing `S` in a listing tells you a decision is coming.

`CMP` is `SUBS` that throws the answer away.

Condition Codes

Code	Meaning	Flags
EQ	Equal	Z=1
NE	Not equal	Z=0
MI	Negative	N=1
PL	Positive or zero	N=0
VS	Overflow	V=1
AL	Always (default)	—

Code	Meaning	Flags
HS/CS	Unsigned $\geq$	C=1
LO/CC	Unsigned $<$	C=0
HI	Unsigned $>$	C=1, Z=0
LS	Unsigned $\leq$	C=0 or Z=1
GE	Signed $\geq$	N=V
LT	Signed $<$	N$\neq$V
GT	Signed $>$	Z=0, N=V
LE	Signed $\leq$	Z=1 or N$\neq$V

Signed and unsigned comparisons are different instructions

GT and HI both mean "greater than." But one reads N and V, and the other reads C and Z. Use a signed code for signed data: for example, $-1 > 5$ is false when signed, but true when unsigned.

Branch Instructions

A branch writes a new value into the Program Counter. This redirects execution. Four forms matter:

Instruction	Meaning
`B label`	Unconditional branch — the basic jump
`BL label`	*Branch with link* — LR = return address, then jump: a subroutine call
`BX Rn`	*Branch and exchange* to an address in a register; `BX LR` returns

Function addresses have their low bit set

The Cortex-M4 executes only Thumb instructions. Function addresses therefore have bit 0 set to mark a valid Thumb target, and `BX`/`BLX` check it. In a debugger, this looks like an off-by-one error, until you know to expect it.

© Muním Zabidi

Table of Contents

1 Why This Matters

2 Making Decisions

3 **Implementing C Control Structures**

4 Subroutines

5 The Stack

6 Procedure Call Standard for the ARM Architecture (AAPCS)

7 Summary

All Control Structures Are Just Branches

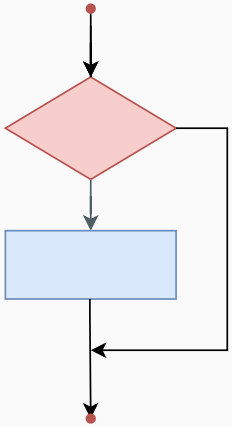
Every high-level control structure you are about to see is built from exactly the two pieces just introduced: a comparison that sets flags, and a conditional branch that tests them. Nothing new is added — only combined.

C construct	Built from
`if`	one conditional branch
`if/else`	one conditional branch **+** one unconditional branch
`while`	one conditional branch **+** one backward branch
`do/while`	one backward conditional branch, no forward branch at all

Keep this map in view through the next few slides: each worked example below is one more arrangement of the same two pieces, never a new mechanism.

© Muním Zabidi

The `if` Construct



"opposites"

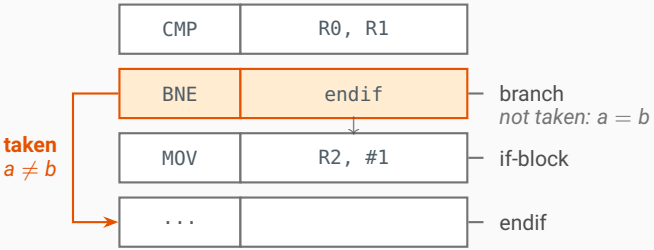
```
if ( a == b )
    x = 1;
```

```
        CMP   R0, R1
        BNE   endif
        MOV   R2, #1
endif
```

Read the branch as the opposite of the C condition: the code tests `x > 0`. The assembly branches away with `BLE` to *skip* the body when this is not true.

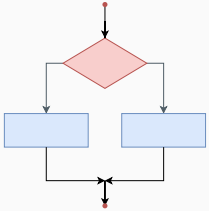
© Muním Zabidi

The Two Paths Through a Plain `if`



The conditional branch tests the *opposite* condition and jumps over the if-block. On $a = b$ the branch is not taken, and execution *falls through* into the block.

The `if-else` Construct



```
if (a == b)
    x = 1;
else
    x = 2;
```

```
                CMP   R0, R1
        BNE   else_block

; if-block: a == b
        MOV   R2, #1
        B     endif

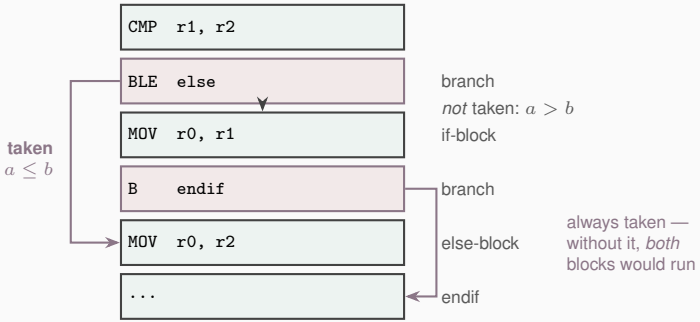
else_block
        ; Else-block: a != b
        MOV   R2, #2          ; x = 2

endif
```

Both arms fall through without `B endif`

The `true` arm must jump past the `else` arm. Without `B endif`, execution falls through and runs both arms.

The Two Paths Through an `if-else`



The conditional branch tests the *opposite* condition and jumps over the if-block; the unconditional branch jumps over the else-block. Execution reaches exactly one `MOV` — never zero, never both.

Greater Than, Less Than BLE and BGE: branch on the opposite

if (x > y)

```
if (x > y) {
    z = x - y;
}
```

```
    CMP   R0, R1    ; x vs y
    BLE   endif     ; x <= y: skip
    SUB   R2, R0, R1
endif
```

if (x < y)

```
if (x < y) {
    x = y;
}
```

```
    CMP   R0, R1    ; x vs y
    BGE   endif     ; x >= y: skip
    MOV   R0, R1    ; x = y
endif
```

The branch asks the opposite question

x > y skips when x <= y, so BLE. x < y skips when x >= y, so BGE. Both are signed branches.

```
if (x > y)
    max = x;
else
    max = y;
```

```
    CMP   R0, R1     ; x vs y
    BLE   elsepart   ; x <= y: else

    MOV   R2, R0     ; max = x
    B     endif      ; skip else

elsepart
    MOV   R2, R1     ; max = y

endif
```

Same pattern as BNE

The conditional branch jumps to the else-block. The unconditional branch skips it. Only the condition code changes.

Every Other `B{cc}` in Three Notes

1. Branch on the opposite. Write the C condition, then branch on its opposite. The opposite pairs are `EQ/NE`, `GT/LE`, `GE/LT`, `HI/LS`, and `HS/LO`.
2. Read the name. `G` is greater, `L` is less, `T` is than, `E` is or equal. So `BLE` is “branch if less than or equal”. For unsigned data, `HI` is higher, `LO` is lower, `S` is same.
3. The data type picks the family. Signed data uses `GT`, `GE`, `LT`, `LE`. Unsigned data uses `HI`, `HS`, `LO`, `LS`.

You now have all of Table 7.1

No separate example is needed for each condition code.

Signed or Unsigned? Same CMP, different branch

Signed: temperature

```
if (temp >= 50) {
    alarm = 1;
}
```

```
    CMP   R0, #50
    BLT   endif     ; temp < 50
    MOV   R1, #1    ; alarm = 1
endif
```

Unsigned: ADC reading

```
if (adc > 2048) {
    high = 1;
}
```

```
    CMP   R0, #2048
    BLS   endif     ; adc <= 2048
    MOV   R1, #1    ; high = 1
endif
```

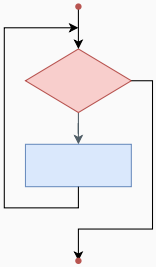
The data type decides the family

An ADC reading is never negative, so it is `uint16_t` and the compiler uses BLS, not BLE.

The `while` Construct

- Implements both the while and for keywords in C
- The condition is tested before the loop starts: a pre-test loop

sum = 1+2+3+4+5



```
sum = 0;
for (i = 1; i <= 5; i++)
    sum = sum + i;
```

```
                MOV   r0, #0
                MOV   r1, #1
loop
            CMP   r1, #5    ; r1 holds i
            BGT   done
            ADD   r0, r0, r1
            ADD   r1, r1, #1
            B     loop
done
```

Worked Example: Popcount

- **Popcount** (population count) means counting the number of bits that are 1

This example is primarily about reusing flags generated by another instruction: LSRS shifts *and* sets flags, so one instruction does the work `while` needed anyway.

```
unsigned int x = 0x5555;   // input
int c = 0;
for (; x != 0; x >>= 1)
    if (x & 1)
        c++;
```

```
                LDR   r1, =0x5555
                MOV   r0, #0        ; count = 0
loop
                AND   r2, r1, #1    ; check bit
     0
                ADD   r0, r2        ; c++
        LSRS  r1, r1, #1    ; next bit
        BNE   loop
done
```

A new principle: one instruction, two jobs

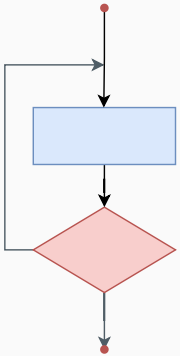
LSRS shifts *and* sets **Z** when the remaining bits reach zero, so the shift itself supplies the condition the next branch needs — no separate comparison. This is the same S-suffix idea from two slides ago, now doing useful work *and* preparing the next decision. Note: if x starts at 0, this loop still runs once.

© Muním Zabidi

The do-while Construct

The condition is tested after the loop body runs: a post-test loop.

sum = 1+2+3+4+5



```
sum = 0;
i = 1;
do {
    sum = sum + i;
    i++;
} while (i <= 5);
```

```
                MOV   r0, #0
                MOV   r1, #1
loop
        ADD   r0, r0, r1
        ADD   r1, r1, #1
        CMP   r1, #5
        BLE   loop
```

A do-while always runs its body at least once. The comparison comes after the work, not before.

Loops: Counting Down Is Cheaper

The flags are not only for comparisons — an arithmetic instruction with the `S` suffix can supply them too, exactly as `LSRS` did in **Popcount**.

Counting up

```
     MOV  r0, #0
loop
     ...          ; body
     ADD  r0, #1
     CMP  r0, #10
     BLT  loop
```

Three instructions of overhead per iteration.

Counting down

```
     MOV  r0, #10
loop
     ...          ; body
     SUBS r0, #1  ; and set flags
     BNE  loop
```

Two — the `CMP` is gone.

Why the `S` suffix saves an instruction

`SUBS` decrements *and* sets **Z** when the result reaches zero, so no separate comparison is needed. This is the most common hand-optimisation in ARM assembly, and compilers do it automatically whenever the loop order does not matter.

Counting Loops with BGE and BLE Leave the loop on the opposite condition

`for` loop: leave on BGE

```
sum = 0;
for (i = 0; i < n; i++) {
    sum = sum + i;
}
```

```
     MOV   R2, #0      ; sum
     MOV   R0, #0      ; i
loop
     CMP   R0, R1      ; i vs n
     BGE   done        ; i >= n
     ADD   R2, R2, R0
     ADD   R0, R0, #1
     B     loop
done
```

Countdown: leave on BLE

```
sum = 0;
while (n > 0) {
    sum = sum + n;
    n--;
}
```

```
     MOV   R1, #0      ; sum
loop
     CMP   R0, #0      ; n vs 0
     BLE   done        ; n <= 0
     ADD   R1, R1, R0
     SUB   R0, R0, #1
     B     loop
done
```

Test at the top

Both loops test before the body, so a loop with `n = 0` does not run at all.

Worked Example: Find Maximum

- Find the biggest signed word value in a memory block, starting at a known address

```
int max = 0;
int *ptr = (int *)0x20001000;
int nwords = 64;
do {
    if (*ptr > max)
        max = *ptr;
    ptr++;
    nwords--;
} while (nwords);
```

```
                MOV     r0, #0          ; max
                LDR     r1, =0x20001000 ; ptr
                MOV     r2, #64         ; nwords
loop
        LDR     r3, [r1], #4    ; *ptr, ptr++
        CMP     r0, r3
        BGE     skip
        MOV     r0, r3          ; new max
skip    SUBS    r2, #1
        BNE     loop
```

`LDR Rn, =value` loads a number, not a variable

`ptr` and `nwords` hold a *fixed* starting address and count, known at compile time. The assembly loads those known values directly into `r1`/`r2` — there is no need to read `ptr`/`nwords` back out of memory, since nothing else in the program ever changes them.

Table of Contents

1 Why This Matters

2 Making Decisions

3 Implementing C Control Structures

4 **Subroutines**

5 The Stack

6 Procedure Call Standard for the ARM Architecture (AAPCS)

7 Summary

Why Subroutines Exist

As programs grow, the same instructions are often needed in more than one place. Rather than copying them, place them in one block and call that block whenever it is needed.

```
uint32_t read_gpioport(void) {
    return GPIOA->IDR;
}
```

A caller simply writes

```
uint32_t state = read_gpioport();
```

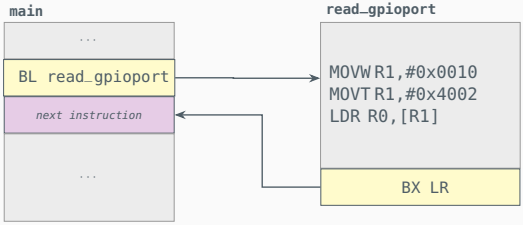
The same idea applies in assembly: the instructions that read the port live in one place, and the program branches there whenever the value is needed — with one new requirement. A subroutine must return to the instruction after the call.

This is how assembly actually shows up in real firmware

Almost nobody writes a whole project in assembly. The usual pattern is C code — `main()` and everything it calls — with one or two hand-tuned routines written in assembly and linked in. `read_gpioport` is exactly that: C calls it like any other function, with no idea its body is assembly.

Calling and Returning: BL and BX LR

- BL branches to a subroutine and records where execution should return.
- The target address is loaded into the PC, while the address of the instruction after BL is placed in LR.
- LR therefore holds the return address for the subroutine.
- Rather than pushing the return address onto the stack, ARM initially keeps it in the LR register.



BL calls a subroutine; BX LR returns from it.

© Muním Zabidi

Returning a Value

How does a subroutine hand a result back? By another convention: the return value goes in R0.

```
read_gpioport
                MOVW  R1, #0x0010   ; GPIOA_IDR low half
                MOVT  R1, #0x4002   ; GPIOA_IDR high half
                LDR   R0, [R1]      ; read GPIO input register
                BX    LR            ; return, with result in R0
```

```
                BL    read_gpioport
                ; R0 now contains the GPIO port value
```

`read_gpioport` takes no arguments, yet still returns a value — the two conventions are independent. The next slide shows `R0` used for an argument too.

Passing Arguments

Now consider a function that needs information *from* its caller:

```
uint32_t read_bit(uint32_t port, uint32_t mask) {
    return port & mask;
}
```

The first four 32-bit arguments travel in `R0`–`R3`:

- `R0` = `port`
- `R1` = `mask`
- `R0` = result, on return

```
read_bit
    AND   R0, R0, R1
    BX    LR
```

`R0` therefore plays two roles at different points in the same call: it carries a value in, as the first argument, and it carries the result out, once the function returns.

`R0`–`R1` for arguments, `R0` for the return value — this is not a rule invented for one function. It is a shared convention every function obeys. Two more leaf subroutines show it holding under slightly different conditions, before asking what happens once one subroutine calls another.

Worked Example: A Value That Must Survive

`read_gpioport` and `counter` share the same shape — no parameters, one result in `R0`, a leaf function — but `counter` needs something `read_gpioport` did not: a value still there the next time it is called.

```c
int counter(void) {
    static int n = 0;
    return ++n;
}
```

```
counter
     LDR   R0, =n
     LDR   R1, [R0]
     ADDS  R1, R1, #1
     STR   R1, [R0]
     MOV   R0, R1
     BX    LR
```

A register cannot hold n

Every register's contents are gone the moment the function returns. `n` must live in memory, at a fixed address, like any global or static variable. The register traffic here only gets that value into `R0` and back out again.

Worked Example: `x = foo(y, z)`

`read_bit` returned the result of a single `AND`. A real function body is rarely that short:

```
int foo(int a, int b) {
    return a * 2 + b;
}

x = foo(y, z);
```

```
foo
     LSLS  R0, R0, #1   ; a*2
     ADDS  R0, R0, R1   ; +b
     BX    LR
```

```
; y already in R0, z in R1
     BL    foo
     MOVS  R4, R0        ; x = result
```

Same convention, more work inside

`a*2` is the shift-instead-of-multiply method from Chapter 6 — the calling convention changes nothing about how the body is optimised. `R0` still carries the argument in and the result out, exactly as `read_bit` showed; only the number of instructions in between has grown.

The Internals Can Change Without the Caller Knowing

Every example so far has had exactly one implementation. Consider a function called exactly like any other:

```
int popcount(uint32_t x);
...
int n = popcount(mask);
```

A straightforward version checks every bit, shifting right until nothing remains — the same loop as the earlier Popcount example, now wrapped as a callable function:

```
popcount
    MOV   R1, #0        ; count = 0
loop
    ANDS  R2, R0, #1    ; check bit 0
    ADD   R1, R1, R2    ; count += bit
    LSRS  R0, R0, #1    ; next bit
    BNE   loop
    MOV   R0, R1
    BX    LR
```

A Faster popcount: Same Signature

One loop iteration per bit position — 32 in the worst case. A well-known method does better: `x & (x - 1)` clears the lowest set bit, so the loop runs once per 1 bit, not once per position.

```
popcount
    MOV   R1, #0        ; count = 0
    CMP   R0, #0
    BEQ   done
loop
    ADD   R1, R1, #1    ; count++
    SUBS  R2, R0, #1    ; R2 = x - 1
    ANDS  R0, R0, R2    ; x &= (x - 1)
    BNE   loop
done
    MOV   R0, R1
    BX    LR
```

Same signature, same calling sequence — different caller cannot tell

Argument in `R0`, result in `R0`, `BX LR` to return — identical to the first version. A function's internals can be rewritten and replaced, and every caller keeps working untouched.

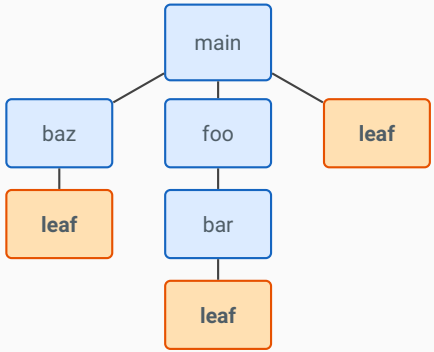
Leaf Versus Non-Leaf Subroutines

- A leaf subroutine does not call another subroutine. It executes no BL, so its LR remains unchanged.
- A non-leaf subroutine calls another subroutine. Its next BL will overwrite the value currently in LR.

This gives us an important rule:

A non-leaf subroutine must save its LR before making another call.

The stack provides a convenient place to save it.

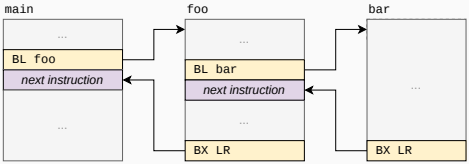


Filled boxes are leaf subroutines. Plain boxes call another subroutine and therefore must preserve LR.

© Munim Zabidi

The Problem: A Second BL Overwrites LR

Consider this three-level call: `main` calls `foo`, and `foo` calls `bar`.



When `main` executes `BL foo`, the return address for `main` is placed in LR.

But then `foo` executes `BL bar`. This places a new return address in the same register, overwriting the original address.

Now `bar` can return to `foo`, because LR contains the correct address for that return. However, the original return address back to `main` has been lost.

One register cannot hold two return addresses.

Spot the Bug This function never returns

`outer` calls `inner` and then returns to its own caller:

```
outer
    BL    inner      ; call inner
    BX    LR         ; return to caller
```

Your prediction

- What does `BL inner` write into LR?
- Where does `BX LR` then jump?
- What is the fix?

Spot the Bug This function never returns

`outer` calls `inner` and then returns to its own caller:

```
outer
    BL    inner      ; call inner
    BX    LR         ; return to caller
```

Your prediction

- What does `BL inner` write into LR?
- Where does `BX LR` then jump?
- What is the fix?

The answer

`BL` overwrites LR with the address of `BX LR`. So `BX LR` jumps to itself, forever. Fix: `PUSH {LR}` first, and `POP {PC}` to return.

Table of Contents

1 Why This Matters

2 Making Decisions

3 Implementing C Control Structures

4 Subroutines

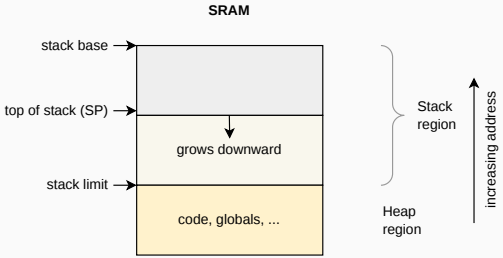
5 **The Stack**

6 Procedure Call Standard for the ARM Architecture (AAPCS)

7 Summary

What Is a Stack?

- A stack is a region of ordinary SRAM managed using the Stack Pointer (SP, R13).
- It follows a last-in, first-out (LIFO) rule: the most recently stored item is the first one removed.
- The stack can be located anywhere in the memory map.
- It is commonly used to hold return addresses, saved registers, and local variables.
- All subroutines typically share the same stack.



- The stack is typically initialized at the top of SRAM.
- Initially, SP points to the top of the stack region.
- As items are pushed, SP moves toward lower addresses.
- The heap, when present, typically grows upward from the opposite direction.

How Does the Stack Grow?

- The stack grows and shrinks by moving the Stack Pointer (SP).
- On a 32-bit Cortex-M processor, each stack word occupies 4 bytes.

PUSH

1. Move SP down by 4 bytes:

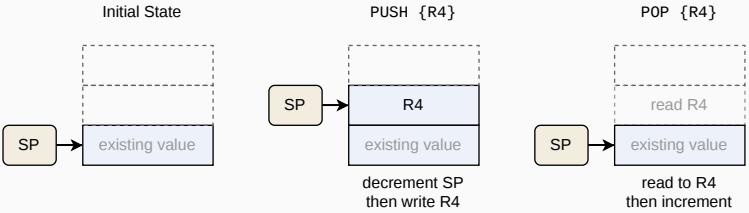
$$SP \leftarrow SP - 4$$

2. Store the value at the new SP address.

POP

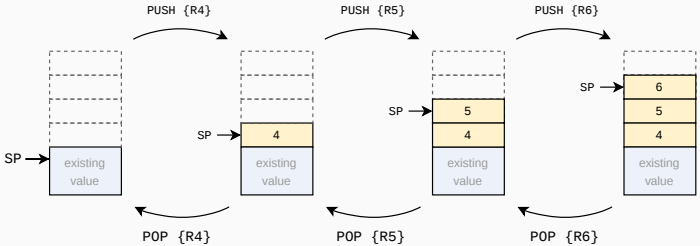
1. Read the value at the current SP address.
2. Move SP up by 4 bytes:

$$SP \leftarrow SP + 4$$



© Muním Zabidi

Why Is It Called Last-In, First-Out?



Three PUSH operations place three registers on the stack. Because each new value is placed on top of the previous one, the first POP retrieves the value pushed last.

The LIFO rule

The last value pushed is the first value popped.

The Stack Must Stay Balanced

- A subroutine may use the stack temporarily to save registers or other data.
- Before returning, it must restore the stack to the state it had on entry.
- In simple terms:

Every PUSH must eventually have a matching POP.

Example

3 registers pushed $\longrightarrow$ SP moves down by $3 \times 4 = 12$ bytes $\longrightarrow$ 3 registers popped $\longrightarrow$ SP returns to its original value

The order matters too: values must be popped in the reverse order in which they were pushed.

What Happens When the Rules Are Broken?

The stack is ordinary memory. If SP moves outside the space or contents intended for the stack, the program can behave unpredictably.

Too many PUSHes

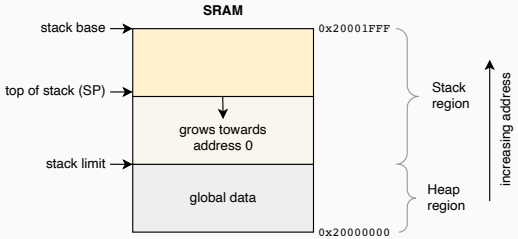
- SP keeps moving toward lower addresses.
- The stack can exceed its allocated region.
- It may collide with other memory regions.
- This is commonly called stack overflow.

Too many POPs

- SP moves beyond the valid stack contents.
- A POP may read data that was never pushed.
- The result may be incorrect data or a fault.
- This is commonly called stack underflow.

© Muním Zabidi

The Stack in Memory It grows downward, in SRAM



On the Cortex-M, the stack is ordinary SRAM. It grows toward lower addresses. A `PUSH` *decrements* `SP`. A `POP` *increments* it. The picture shows the STM32F446RE: 128 KiB of SRAM from `0x20000000` to `0x2001FFFF`.

Where SP starts

`SP = 0x20020000`, just above the top of SRAM. The first `PUSH` lowers it to `0x2001FFFC`, which is valid SRAM. The stack must stay inside its region, or it corrupts other data.

© Muním Zabidi

Saving the Return Address

```
                BL    AddNumbers

AddNumbers
                PUSH  {LR}      ; save caller's return
    address
                ADD   R0, R0, R1
                POP   {PC}      ; restore and return
```

Before making another call, a non-leaf subroutine can move its return address from LR to the stack:

```
PUSH {LR}
```

This preserves the address even when a later BL overwrites LR.

At the end, `POP {PC}` restores the saved address directly into the program counter, returning execution to the caller.

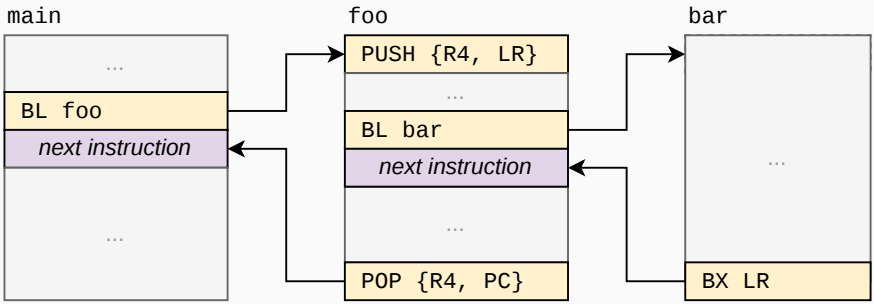
The key idea

LR is convenient for a single call, but it cannot preserve multiple return addresses.

The stack turns one temporary register into persistent storage for the return address.

A leaf subroutine does not need this step: because it makes no further call, its LR remains intact and it can return with `BX LR`.

The Stack Can Save More Than LR



PUSH can save several registers in one instruction. For example, `PUSH {R4, LR}` saves both R4 and the return address on the stack.

When the subroutine finishes, the same registers can be restored with `POP {R4, PC}`.

Restoring LR into PC returns execution to the caller, while restoring R4 returns it to its original value.

Restoring the Return Address POP can load straight into the PC

A function that calls another must first save its own return address, because the nested BL overwrites LR. POP can load directly into PC. So one instruction can restore the registers and return:

```
ProcessData
    PUSH  {R4, LR}     ; save registers
    ; ... body ...
    BL    AddNumbers
    POP   {R4, PC}     ; restore registers and return
```

A return in disguise

This pattern appears often in compiler output. When you read a disassembly, treat `POP {..., PC}` as a return.

Table of Contents

1 Why This Matters

2 Making Decisions

3 Implementing C Control Structures

4 Subroutines

5 The Stack

6 **Procedure Call Standard for the ARM Architecture (AAPCS)**

7 Summary

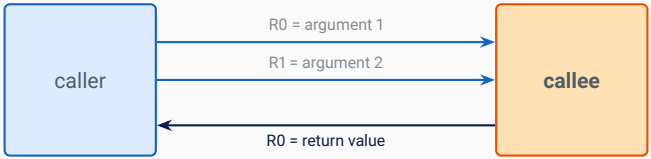
Why a Call Needs More Than Registers

The examples so far passed arguments in `R0`/`R1` and returned a result in `R0`, by convention. The nested-call problem just solved — save `LR` before a second `BL` — raises the same question one level up: which registers are safe to overwrite, and which must a callee restore?

Every function you write, and every library function you call, needs the *same* answer to that question — otherwise C code and hand-written assembly could never call each other safely. That shared answer is a formal convention: the AAPCS.

How Do Two Functions Agree on Anything?

Your C code and a library compiled years ago must agree on where arguments go, where results return, and who preserves what — with no run-time negotiation.



Procedure Call Standard for the ARM Architecture (AAPCS)

The AAPCS is that agreement

It defines where arguments and return values go and which registers a function must preserve. This lets C and assembly functions call each other safely.

Registers	Rule
`R0–R3`	Arguments 1–4; `R0` also carries the return value. Caller-saved: the callee may overwrite them.
`R4–R11`	Callee-saved: the callee must restore any it uses.
`R12`	Scratch register.
`R13–R15`	SP, LR, PC.

Arguments beyond the fourth go on the stack — the same mechanism just used to save LR.

© Muním Zabidi

Do No Damage: Callee-Saved Registers

A function may use R4–R11, but it must restore them before returning. The pattern is the same as saving LR: save, use, restore.

```
foo  PUSH {R4, R5}    ; save registers this function will use
     MOV  R4, #0
     MOV  R5, #0xAB
     ADD  R4, R4, R5
     ...
     POP  {R4, R5}    ; restore their original values
     BX   LR
```

Why this matters

The caller can continue using R4 and R5 after `foo` returns, knowing their values are unchanged. No special agreement is needed at each call site.

More Than Four Arguments The extras go on the stack

```c
int weighted_sum(int a, int b,
                 int c, int d,
                 int e) {
    return a+b+c+d+e;
}
```

The first four arguments use `R0–R3`. The caller puts the rest on the stack. The AAPCS keeps the stack 8-byte aligned at every call.

```asm
; Caller: a..d already in R0-R3
    SUB   SP, SP, #8   ; reserve 8 bytes: stays aligned
    LDR   R12, =e_value ; R12 is scratch
    LDR   R12, [R12]
    STR   R12, [SP]    ; the fifth argument, e
    BL    weighted_sum
    ADD   SP, SP, #8   ; caller releases it

weighted_sum
    ADD   R0, R0, R1   ; a + b
    ADD   R0, R0, R2   ; + c
    ADD   R0, R0, R3   ; + d
    LDR   R1, [SP]     ; e, read from the stack
    ADD   R0, R0, R1   ; + e
    BX    LR
```

Reading a Compiled Function A C function and its disassembly

```c
int sum_array(const int *data,
              int n) {
    int total = 0;
    for (int i = 0; i < n; i++)
        total += data[i];
    return total;
}
```

A compiler turns it into roughly the code on the right. The next slide shows where the AAPCS rules appear in it.

```
sum_array
    PUSH  {R4, LR}    ; prologue: save R4, LR
    MOV   R2, #0      ; total = 0
    MOV   R3, #0      ; i = 0
    CMP   R1, #0      ; is n <= 0 ?
    BLE   done        ; yes: skip the loop
loop
    LDR   R4, [R0, R3, LSL #2] ; data[i]
    ADD   R2, R2, R4           ; total += R4
    ADD   R3, R3, #1           ; i++
    CMP   R3, R1               ; i < n ?
    BLT   loop                 ; signed compare
done
    MOV   R0, R2      ; return value in R0
    POP   {R4, PC}    ; epilogue: restore, return
```

Where the AAPCS Shows Up in `sum_array`

In the code	The rule behind it
`R0`, `R1` used as inputs	The first two arguments arrive in `R0` and `R1`.
`MOV R0, R2` before the return	The result is returned in `R0`.
`R2`, `R3` used freely	Caller-saved scratch registers: the function may destroy them.
`PUSH {R4, ...}` / `POP {R4, ...}`	`R4` is callee-saved, so the function must restore it.
`LR` saved with `R4`	The function must keep its own return address.
`POP {R4, PC}`	Restores `R4` and returns in one step.

The `LDR` uses a scaled register offset, so the index becomes a byte offset inside the load. The compiler emits `BLT` because `i` and `n` are signed. For `unsigned` it would emit `BLO`. The first `CMP R1, #0` handles a loop that runs zero times.

A Function Call Viewed at Five Levels

Level	Representation
Physical hardware	SP drives the address bus; SRAM cells hold the saved LR
Register	`R0`, `R1` carry the arguments; `LR` holds the return address
C	`sum = AddNumbers(5, 10);`
Assembly (call)	`MOV R0, #5` / `MOV R1, #10` / `BL AddNumbers`
Assembly (callee)	`ADD R0, R0, R1` / `BX LR` (a leaf: LR is never saved)

One call, five simultaneous descriptions

Chapters 1–7 have all built this table. The C statement reaches a hardware SRAM cell, through registers, instructions, and bytes.

© Muním Zabidi

Table of Contents

1 Why This Matters

2 Making Decisions

3 Implementing C Control Structures

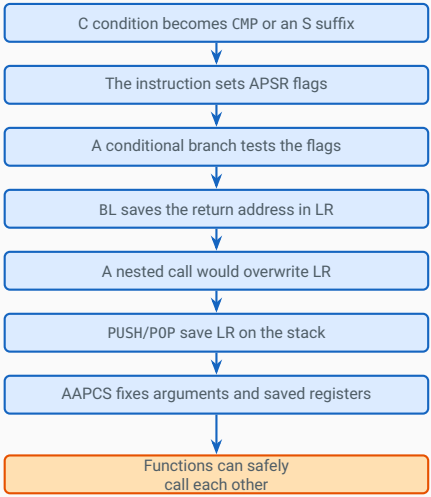
4 Subroutines

5 The Stack

6 Procedure Call Standard for the ARM Architecture (AAPCS)

7 **Summary**

Summary



Terminology covered

- APSR flags, condition codes
- Signed vs unsigned suffixes
- BL, BX LR
- Full descending stack
- AAPCS

Next: Chapter 8

General Purpose Input Output

The book stops being theory. A physical pin changes voltage.

© Muním Zabidi